

Reloj digital

Adrián Macías Mendoza
Unidad Académica de Ciencias Básicas e Ingenierías
Universidad Autónoma de Nayarit
Tepic, Nayarit
adrian.macias@uan.edu.mx

Isaac Macías Mendoza
COCYTEN
Tepic, Nayarit
usuario_cocytan_37_x@cinvestav.mx

Berenice Del Rosario Maldonado Fregoso
Unidad Académica de Ciencias Básicas e Ingenierías
Universidad Autónoma de Nayarit
Tepic, Nayarit
berenice.maldonado@uan.edu.mx

Daniel Armando Ríos Rivera
Universidad Politécnica del Estado de Nayarit
División de Tecnologías de la Información y Comunicación
Tepic, México
daniel.rios@upnay.edu.mx

I. MARCO TEÓRICO

Un reloj digital es un dispositivo electrónico que muestra la hora en formato numérico, a diferencia de los relojes analógicos que utilizan manecillas. Los relojes digitales se basan en circuitos electrónicos capaces de contar y mostrar unidades de tiempo como segundos, minutos y horas. Su precisión depende del componente oscilador que marca los intervalos de tiempo.

El circuito de un reloj digital se compone principalmente de los siguientes bloques funcionales:

- Oscilador: genera una señal de reloj estable (por ejemplo, 1 Hz para un pulso por segundo).
- Divisores de frecuencia: reducen la frecuencia del oscilador a la adecuada para marcar los segundos.
- Contadores: cuentan los pulsos para acumular segundos, minutos y horas.
- Decodificadores: convierten la salida binaria de los contadores a un formato entendible por los displays.
- Displays de 7 segmentos o LCD: muestran los números de la hora al usuario

El reloj digital utiliza un oscilador de cuarzo o un circuito temporizador que produce una señal periódica. Esta señal se divide hasta generar un pulso de un segundo, que alimenta un contador de segundos. Cada 60 pulsos de un segundo, el contador de minutos se incrementa en uno, y así sucesivamente con las horas.

II. ESPECIFICACIONES DEL CIRCUITO

El circuito diseñado para el reloj digital cuenta con 3 señales de entrada clk, rstn y start_stop de 1 bit y 4 salidas hex0, hex1, hex2 y hex3 de 7 bits cada una. Estas se detallan en la siguiente tabla:

Con base en el pinout se detallan las siguientes especificaciones:

- El reloj mostrará la hora (segundos y minutos) cuando la señal de start_stop se encuentre en alto.

Entrada/Salida	Nombre de la señal	Descripción
Input	clk	Señal de reloj del circuito
Input	rstn	Señal de reset asíncrona
Input	start_stop	Señal de inicio y paro del conteo
Output	hex0	Salida de display 7 segmentos para las unidades de segundo
Output	hex1	Salida de display 7 segmentos para las decenas de segundo
Output	hex2	Salida de display 7 segmentos para las unidades de minuto
Output	hex3	Salida de display 7 segmentos para las decenas de minuto

Tabla I
PINOUT DEL RELOJ DIGITAL

- La hora se mostrará en 4 displays de 7 segmentos correspondientes a unidades de segundo, decenas de segundos, unidades de minutos y decenas de minutos.
- Las salidas para el 7 segmentos de las unidades de segundo, hex0, dependerá del valor decimal de la señal interna sec_units, tal y como se muestra en la Tabla II.
- La señal interna sec_units incrementará una unidad desde el 0 hasta el 9 cada 2 ciclos de reloj, siempre y cuando la señal start_stop esté en alto. La cantidad de ciclos es parametrizable pero fue trabajada de esta forma durante el desarrollo.
- Cuando la señal interna sec_units está en 9, esta regresará a 0 en su siguiente incremento.
- Las salidas para el 7 segmentos de las decenas de segundo, hex1, dependerá del valor decimal de la señal interna sec_tens, tal y como se muestra en la Tabla II.
- La señal interna sec_tens incrementará una unidad desde

el 0 hasta el 5 cada que la señal sec_units regrese a 0 después de estar en 9.

- Cuando la señal interna sec_tens está en 5, esta regresará a 0 en su siguiente incremento.
- Las salidas para el 7 segmentos de las unidades de minuto, hex2, dependerán del valor decimal de la señal interna min_units, tal y como se muestra en la Tabla II.
- La señal interna min_units incrementará en una unidad desde el 0 hasta el 9 cada que la señal sec_tens regrese a 0 después de estar en 5.
- Cuando la señal interna min_units está en 9, esta regresará a 0 en su siguiente incremento.
- Las salidas para el 7 segmentos de las decenas de minuto, hex3, dependerán del valor decimal de la señal interna min_tens, tal y como se muestra en la Tabla II.
- La señal interna min_tens incrementará en una unidad desde el 0 hasta el 6 cada que la señal min_units regrese a 0 después de estar en 9.
- Cuando la señal interna min_tens está en 6, esta regresará a 0 en su siguiente incremento.
- La señal de reset asíncrono, rstn, se activará con el flanco de bajada, y hará que la salida de todos los displays sea 1000000, correspondiente a un 0 de sus señales internas correspondientes.

Valor de señal interna	Salida del Display 7 segmentos
0	1000000
1	1111001
2	0100100
3	0110000
4	0011001
5	0010010
6	0000010
7	1111000
8	0000000
9	0011000

Tabla II

SALIDA DE LOS DISPLAYS DE 7 SEGMENTOS SEGÚN EL VALOR DECIMAL DE SU SALIDA INTERNA CORRESPONDIENTE.

La señal interna en la Tabla II hace referencia a las señales internas sec_units, sec_tens, min_units y min_tens que corresponden a los displays hex0, hex1, hex2 y hex3, respectivamente.

III. MICROARQUITECTURA

Para el desarrollo de este circuito se decidió utilizar un divisor de frecuencia por medio de un contador, lo cual permitirá obtener ciclos de reloj lentos a partir de los ciclos veloces que manejan los circuitos electrónicos. Este ciclo de reloj más lento obtenido se dará cada dos ciclos de la señal de reloj principal.

Con la señal más lenta obtenida del divisor se realizó el conteo principal, y cada flanco de subida de este incrementará en uno la señal interna que cuenta las unidades de segundo. A partir de este mismo contador se obtienen las demas señales internas para unidades de minuto y decenas de minuto y segundo.

Las señales internas obtenidas del contador pasarán por un codificador BCD que mostrará el valor decimal en un display de 7 segmentos de ánodo común.

La señal start_stop servirá como un enable que permitirá que la cuenta del reloj se siga llevando a cabo.

La Figura 1 muestra la microarquitectura del circuito implementado, mostrando los componentes descritos den esta sección.

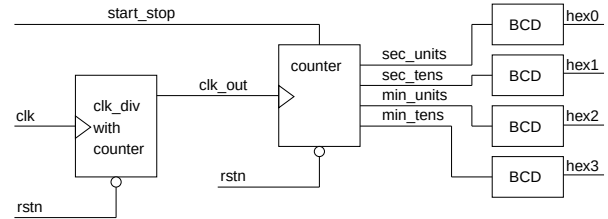


Figura 1. Microarquitectura del reloj digital implementado

IV. MODELO COMPORTAMENTAL

Para realizar el RTL se realizaron 3 módulos con diferentes archivos .sv. El del divisor de frecuencia, el del decodificador BCD y uno principal del reloj que funciona como top y que es donde se instancian los otros dos módulos.

La captura del codigo RTL para el BCD se muestra en la figura 2. La captura del código RTL para el divisor de frecuencia se muestra en la figura 3. La captura del código RTL principal del reloj digital se muestra en la figura 4.

```

1 module bcd_to_7seg(
2     input [3:0] bcd, // Las entradas a esto serán las unidades o
3                       // decenas de seg o min del 0 a 9 (4 bits son suficientes)
4     output reg [6:0] seg // La salida es un display de 7 segmentos
5 );
6 always @(*) begin
7     case (bcd) // Para la entrada bcd y con un display anodo común
8         4'd0: seg = 7'b1000000; // 0x40 (Así se ven normalmente en la sim)
9         4'd1: seg = 7'b1111001; // 0x79
10        4'd2: seg = 7'b0100100; // 0x24
11        4'd3: seg = 7'b0110000; // 0x30
12        4'd4: seg = 7'b0011001; // 0x19
13        4'd5: seg = 7'b0010010; // 0x12
14        4'd6: seg = 7'b0000010; // 0x02
15        4'd7: seg = 7'b1111000; // 0x78
16        4'd8: seg = 7'b0000000; // 0x00
17        4'd9: seg = 7'b0011000; // 0x18
18        default: seg = 7'b1111111; // 0xFF (apagado)
19    endcase
20 end
21 endmodule

```

Figura 2. Código de system verilog para el decodificador BCD

V. RESULTADOS

El resultado de la síntesis realizada por VIVADO del código mostrado en la seccion anterior es el esquemático que se muestra en la Figura 5.

VI. CONCLUSIÓN

Se desarrolló un circuito para un reloj digital que muestra minutos y segundos a través de 4 dislplays de 7 segmentos al activar la entrada de start. El segundero aumenta cada 2 ciclos

```

1 module clk_div(
2     input wire clk_in, // Señal de reloj entrante, propia del sistema
3     input wire rstn,   // Señal de reset
4     output reg clk_out // Señal de reloj saliente
5 );
6     parameter DIVISOR = 2; // Parametro division de señal
7     reg [25:0] count = 0; // Tamaño para 50M (Tomando la DE115 como base)
8
9     always @(posedge clk_in, negedge rstn) begin
10         if (!rstn) begin // Cuando el reset se activa
11             count <= 0; // La cuenta se regresa
12             clk_out <= 0; // Y la señal de salida se pone en bajo
13         end else begin // Si reset no se activa
14             count <= count + 1; // Sumar uno a la cuenta
15             if (count == DIVISOR/2 - 1) begin // Si llega al final
16                 clk_out <= ~clk_out; // Cambiamos el valor de la señal de salida
17                 count <= 0; // Reseteamos la cuenta
18             end
19         end
20     end
21 endmodule

```

Figura 3. Código de system verilog para el divisor de frecuencia

```

1 module digital_clock(
2     input wire clk, // Señal de reloj
3     input wire rstn, // Señal de reset
4     input wire start_stop, // Señal de activación
5     output wire [6:0] hex0, // Unidades de segundo
6     output wire [6:0] hex1, // Decenas de segundo
7     output wire [6:0] hex2, // Unidades de minuto
8     output wire [6:0] hex3 // Decenas de minuto
9 );
10
11 wire clk_signal; // Señal de reloj para el conteo
12
13 /* Genera una señal de reloj a partir del reloj base
14 y un parámetro de conteo de ciclos de reloj*/
15 clk_div clk_div_1 (
16     .clk_in(clk), // Entre la señal del reloj original
17     .rstn(rstn), // Señal de reset
18     .clk_out(clk_signal) // Señal de frecuencia reducida
19 );
20
21 // Registros para segundos y minutos
22 reg [5:0] seconds = 0; // 6 bits para contar hasta 59
23 reg [5:0] minutes = 0; // 6 bits para contar hasta 59
24
25 always @(posedge clk_signal, negedge rstn) begin
26     if (!rstn) begin // Regresa el conteo a 0
27         seconds <= 0;
28         minutes <= 0;
29     end else if (start_stop) begin // funciona como un enable
30         if (seconds == 59) begin // hace conteo hasta llegar al último valor
31             seconds <= 0; // y resetea
32             if (minutes == 59) // Es necesario preguntar cual es el conteo de min
33                 minutes <= 0; // Si este llega al final hay que resetear tambien
34             else
35                 minutes <= minutes + 1; // Si si no ha llegado al final sumeter 1 al conteo
36         end else begin
37             seconds <= seconds + 1; // Si no ha ocurrido nada de lo anterior sumar a seg
38         end
39     end else if (!start_stop) begin //
40         seconds <= seconds; // La cuenta se debe mantener
41         minutes <= minutes; // igual aqui
42     end
43 end
44
45 // Conversión a dígitos BCD
46 wire [3:0] sec_units = seconds / 10; // el residuo de la div entre 10 la cuenta
47 wire [3:0] sec_tens = seconds % 10; // la división entre diez dara las decenas de seg
48 wire [3:0] min_units = minutes / 10; // Misma logica para unidades de minutos
49 wire [3:0] min_tens = minutes % 10; // E igual para decenas de minutos
50
51 // Decodificadores BCD a 7 segmentos
52 bcd_to_7seg bcd0 (.bcd(sec_units), .seg(hex0)); // segundos unidades
53 bcd_to_7seg bcd1 (.bcd(sec_tens), .seg(hex1)); // segundos decenas
54 bcd_to_7seg bcd2 (.bcd(min_units), .seg(hex2)); // minutos unidades
55 bcd_to_7seg bcd3 (.bcd(min_tens), .seg(hex3)); // minutos decenas
56
57 endmodule

```

Figura 4. Código principal de system verilog para el reloj digital

de reloj, aunque el parámetro pertinente se puede modificar para que se ajuste a distintas frecuencias del ciclo de reloj.

Durante el proceso fue complicado ajustarse a la metodología preferente de realizar la documentación primero. Notamos que tuvimos que realizar algunos cambios en nuestro RTL para ajustarse a una documentación clara y fácil de entender para aquellos no involucrados en el proceso de realización. Si se hubiera realizado la documentación primero, pudimos haber ahorrado tiempo durante la realización del código.

Trabajar con el equipo de verificación también nos mostró

un poco de como se trabaja en la industria, y de como el trabajo colaborativo logra grandes resultados, ya que fue a través de ellos como se encontraron muchos de los detalles que tuvieron que mejorarse, sin olvidar la ayuda de nuestro instructor.

BIBLIOGRAFÍA

- [1] F. J. Rodríguez Navarrete. (2025). Sequential logic modelling I [PowerPoint slides].
- [2] F. J. Rodríguez Navarrete. (2025). Sequential logic modelling II [PowerPoint slides].
- [3] IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language, in IEEE Std 1800-2017 (Revision of IEEE Std 1800-2012), vol., no., pp.1-1315, 22 Feb. 2018, doi: 10.1109/IEEESTD.2018.8299595.

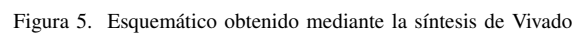


Figura 5. Esquemático obtenido mediante la síntesis de Vivado